

Уязвимости веб-приложений (конспект).

1. SQL-инъекция.

SQL -инъекция - внедрение вредоносного кода в запрос к БД.

Пример: - вставка в форму SQL-кода в одинарных кавычках.

Как защититься:

- подготовленные запросы с масками (\$db->prepare('... WHERE id=:id ...'));
- для имён таблиц и столбцов используйте "белый список" (список разрешенных вещей, когда по умолчанию все запрещено);
- валидируйте загрузку файлов (определять mime-type), пересохраняйте изображения.

2. XSS (межсайтовый скриптинг).

XSS атака - внедрение вредоносного кода на страницу сайта.

Пример: - вставка кода в форму (например, тега "script" со своим кодом); - вставка кода в адресную строку.

Две главные разновидности:

- 1) Первого порядка - выполняется немедленно;
- 2) Второго порядка - пишется в базу, выполняется потом.

Как защититься:

- Обработать пользовательский ввод. Например, с помощью функций strip_tags(), htmlspecialchars().
- Если нужен HTML от пользователя, то используем библиотеку `HTMLPurifier` (<http://htmlpurifier.org/>).
- Не выводить на странице необработанные GET-параметры.
- Отправлять заголовки для обозначения допустимых ресурсов.
- Использовать мета-теги, блокирующие обращение на сторонние ресурсы.
- Использовать куки-параметр `HttpOnly` (запрет JavaScript-у использовать куки).

3. Утечки информации.

- Включенный отладочный режим.
- Страницы ошибок.
- Ответы API.
- ...

Сообщение об ошибках могут дать информацию, пригодную для взлома.

Пример:

- запустить отладочный механизм, подбирая основные признаки отладки:

```
<form method="post" action="attacktarget?errors=Y&showerrors=1&debug=1">
  <input type="hidden" name="errors" value="Y" />
  <input type="hidden" name="showerrors" value="1" />
  <input type="hidden" name="debug" value="1" />
</form>
```

Как защититься:

- использование функции `error_reporting()`.

4. CSRF / XSRF.

Cross Site Request Forgery — подделка межсайтовых запросов.

Пример: - отправка формы с сайта злоумышленника на другой сайт от имени авторизованного пользователя.

Как защититься: - Использовать CSRF-токены (один и тот же в форме и в сессии). - Ипользовать SSL-сертификаты. - Не использовать GET для изменения состояния приложения:

```

// Использование $_REQUEST - то же самое
```

5. Инъекция кода.

Например, если используется `eval()`

Как защититься: - не использовать `eval()` или использовать после проверки по "белому списку"

6. Небезопасные `include`

Пример:

```
require $_GET['type'] . '.php';
```

Как защититься: - Если нельзя убрать такие `include`, то использовать "белый список"

7. Clickjacking (Взлом кликов).

Обманом заставить пользователя кликнуть на что-то на строннем сайте.

Пример: - с помощью `iframe` + `opacity:0` подсунуть интерфейс другого сайта.

Как защититься: - запретить записывать сайт в `iframe`:

```
header('X-Frame-Options: DENY'); // запретить совсем
header('X-Frame-Options: SAMEORIGIN'); // разрешить только на своем домене
// или через JavaScript ...
```

8. Пароли.

Злоумышленник может угадать ваш пароль - перебором, - атакой по словарю, - атакой на основе времени ответа. Пароли могут "утечь" вместе с БД.

Есть два параметра для пароля: - Cost - стоимость вычисления хеша (больше 11 - безопасное значение на данный момент) - сложность пароля

Как защититься: - ограничить попытки: 10 попыток с IP в минуту или CAPTCHA (ненадежно); - вы не должны знать пароль пользователя. Храните хэш; - для хэша не использовать функции `md5`, `sha1`, `sha256`, `sha512` и т.д.; - для хэша использовать как минимум `bcrypt`; - для генерации случайных чисел использовать: - PHP7 `random_bytes` - LibreSSL - `CryptGenRandom` (Windows) - `/dev/random` (FreeBSD) - `/dev/urandom` (Unix) - Если ничего не получилось, кидаем исключение.

9. Фиксация сессии.

Пример: - Дают пользователю заранее известный ID сессии. - Перехватывают cookie.

Как защититься: - использовать только куки. В `php.ini` прописать:

```
session.use_cookies = 1
session.use_only_cookies = 1
session_cookie_httponly = 1
session.cookie_secure = 1
```

- регенирировать ID сессии при помощи `session_regenerate_id(true)` после логина или смены прав.

Дополнительные рекомендации.

- Не сохраняйте экранированные данные в базу. Они могут вам понадобиться в исходном виде. К тому же, базе доверять нельзя.
- По возможности запускайте всё в контейнерах (Docker или др.).
- Запускайте PHP с минимально возможными правами.
- Не забывайте проверять доступ (например, к файлу по URL).
- Сначала запрещайте всё, потом уже разрешайте.
- Обновляйтесь (Linux, Nginx, PHP, СУБД, ...).
- Помните, что зависимости могут быть с уязвимостями.
- `target="_blank"` не безопасен.
- ImageMagick не безопасен.
- SMS не безопасны (лучше Push уведомления), например, для 2-хфакторной авторизации.
- XML не безопасны (атака XXE - инъекция внешних сущностей).
- Десериализовать опасно.
- Процессоры не безопасны.

Полезные ссылки на данную тему.

- <https://www.youtube.com/watch?v=R0pMnVLIX10> - WEB - безопасность: От базовых принципов до особенностей PHP - Александр Макаров
- <https://www.php.net/manual/ru/security.php> - Руководство по PHP. Безопасность.

Почитать.

- OWASP + testing guide
- Фаулер про безопасность
- Компонент Yii security
- QA на StackExchange